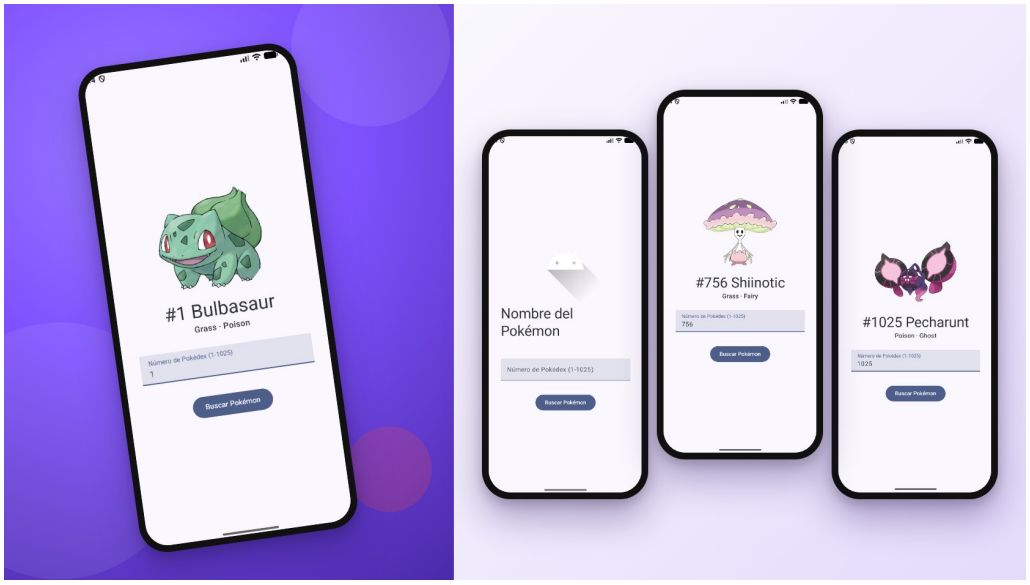


Pokédex Kotlin

Eva Gallardo Romero

23 de enero de 2026 a 30 de enero de 2026

Índice

Enlace al repositorio de GitHub **2**

1. Descripción general **2**

2. Diagrama de clases **2**

3. Diseño **2**
3.1. Antes de buscar 2
3.2. Resultado de la búsqueda 3
3.3. Número no encontrado 4

4. Requisitos del sistema **5**
4.1. Requisitos funcionales 5
4.2. Requisitos no funcionales 5

5. Documentación técnica **5**
5.1. Pokemon.kt 6
5.1.1. Ubicación 6
5.1.2. Funciones 6
5.2. API.kt 6
5.2.1. Ubicación 6
5.2.2. Funciones 6
5.3. Buscador.kt 7
5.3.1. Ubicación 7
5.3.2. Funciones 7

6. Mejoras futuras **9**

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo y descargable en el siguiente enlace: PokedexKotlin.

La aplicación instalable (APK) está en la sección Releases.

1. Descripción general

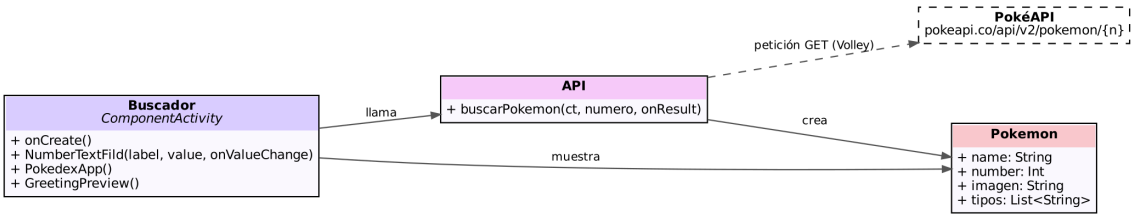
Pokédex Kotlin es un **buscador de Pokémon** para Android hecho con **Kotlin** y **Jetpack Compose**. El usuario escribe el **número de la Pokédex** (del 1 al 1025), pulsa **Buscar Pokémon** y la app consulta la PokéAPI para mostrar su **ilustración oficial**, su **nombre** y sus **tipos**.

Es la versión en Compose de mi Pokédex en Java, hecha con vistas XML. Tenerlas juntas me sirve para comparar cómo se resuelve lo mismo con una **interfaz declarativa** y con el sistema clásico de vistas.

La hice en clase entre el 23 y el 30 de enero de 2026 para practicar las funciones `@Composable`, el **estado** con `remember` y `mutableStateOf`, los componentes de **Material 3** y las **llamadas a una API REST**.

2. Diagrama de clases

La app tiene tres clases: la pantalla (`Buscador`), la que habla con la PokéAPI (`API`) y el modelo (`Pokemon`).

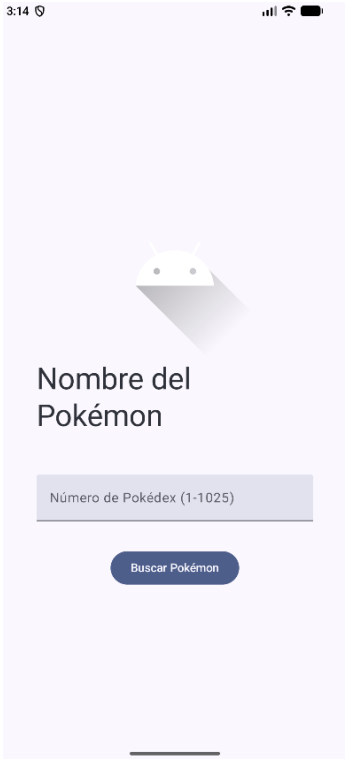


3. Diseño

La app tiene una sola pantalla, con los colores por defecto de **Material 3**: la imagen arriba, el nombre y los tipos debajo, el **campo del número** y el **botón** de búsqueda.

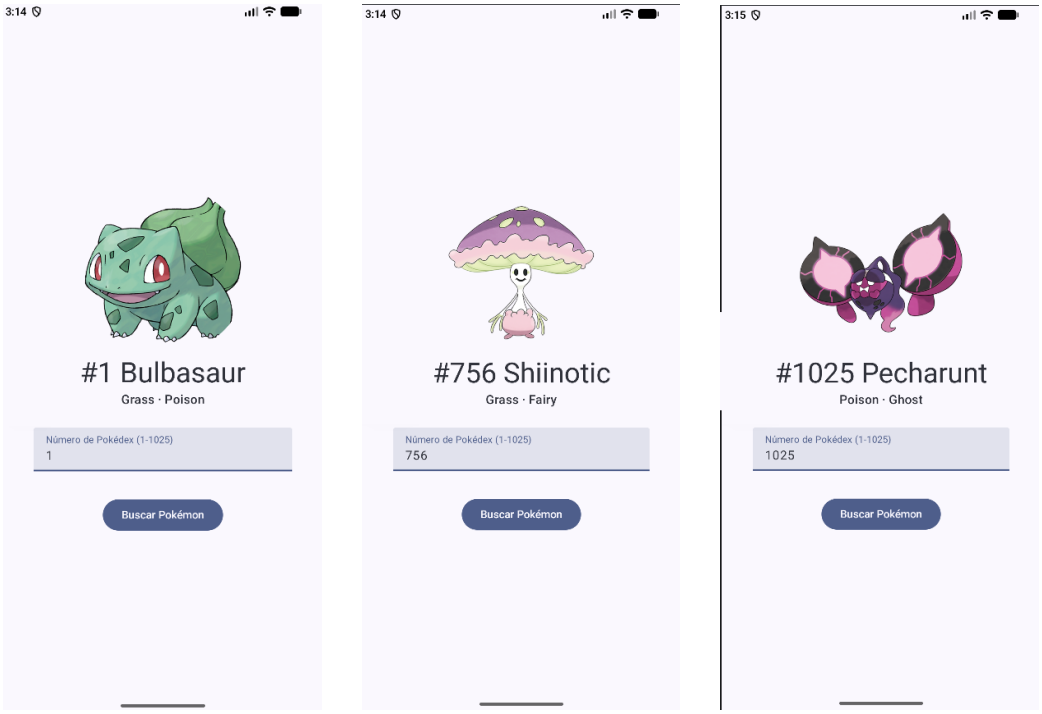
3.1. Antes de buscar

Al abrir la app se ve la imagen de reserva y el texto «*Nombre del Pokémon*». El campo abre directamente el **teclado numérico**.



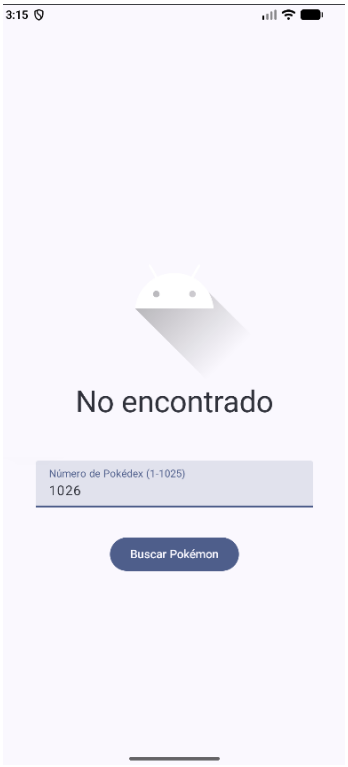
3.2. Resultado de la búsqueda

Cuando llega la respuesta, la app muestra la **ilustración oficial**, el **número y el nombre** (por ejemplo, «*#1 Bulbasaur*») y los **tipos** separados por un punto. Mientras espera, el botón se desactiva y muestra «*Buscando…*».



3.3. Número no encontrado

Si el número no existe en la PokéAPI (por ejemplo, el 1026) o no hay conexión, la app muestra «*No encontrado*». Si el campo está vacío o el número no es válido, pide «*Escribe un número válido*».



4. Requisitos del sistema

4.1. Requisitos funcionales

ID	Requisito funcional
RF01	El usuario debe poder **escribir un número de la Pokédex** con el teclado numérico.
RF02	La aplicación debe **validar el número** y avisar si está vacío o no es válido.
RF03	El sistema debe **consultar la PokéAPI** con una sola petición por búsqueda.
RF04	La aplicación debe mostrar la **ilustración oficial** del Pokémon.
RF05	La aplicación debe mostrar el **número y el nombre** del Pokémon.
RF06	La aplicación debe mostrar los **tipos** del Pokémon.
RF07	Mientras busca, el botón debe **desactivarse** y mostrar «Buscando...».
RF08	El sistema debe avisar con «**No encontrado**» si el Pokémon no existe o falla la conexión.

4.2. Requisitos no funcionales

ID	Requisito no funcional
RNF01	La interfaz debe estar hecha con **Jetpack Compose** y **Material 3**.
RNF02	La aplicación necesita **conexión a internet** (permiso `INTERNET`).
RNF03	Las imágenes se deben **descargar y guardar en caché** con Coil.
RNF04	La aplicación **no debe cerrarse** si la respuesta falla o no es la esperada.
RNF05	La pantalla debe **adaptarse a distintos tamaños** de móvil.
RNF06	La aplicación debe funcionar en **Android 7.0 (API 24)** o superior.

5. Documentación técnica

Este anexo agrupa la documentación por archivos, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de ma-

nera individual con su **código** adjunto y comentado.

5.1. Pokemon.kt

5.1.1. Ubicación

`com.example.pokedexkotlin`

Descripción general:
La clase `Pokemon` es el modelo con los datos que muestra la app: el nombre, el número de la Pokédex, la URL de la ilustración oficial y la lista de tipos.

5.1.2. Funciones

Clase **Descripción:** Declara los cuatro atributos en el constructor principal. Los tipos tienen como valor por defecto una lista vacía.

Código comentado:

```
class Pokemon (
    var name : String,
    var number : Int,
    var imagen : String,
    var tipos : List<String> = emptyList()
)
```

5.2. API.kt

5.2.1. Ubicación

`com.example.pokedexkotlin`

Descripción general:
La clase `API` hace la petición a la PokéAPI con **Volley** y convierte el JSON de respuesta en un objeto `Pokemon`.

5.2.2. Funciones

buscarPokemon **Descripción:** Pide `pokeapi.co/api/v2/pokemon/{numero}`, saca del JSON el nombre, el número, la ilustración oficial y los tipos, y devuelve el `Pokemon` por la función `onResult`. Si el número no existe, no hay conexión o el JSON no es el esperado, devuelve `null`.

Código comentado:

```
// Busca un Pokémon por su número de la Pokédex en la PokéAPI.
// Cuando llega la respuesta llama a onResult con el Pokémon, o con null si no existe
// o si no hay conexión.
fun buscarPokemon(ct: Context, numero: Int, onResult: (Pokemon?) -> Unit) {
    val queue: RequestQueue = Volley.newRequestQueue(ct)
    val url: String = "https://pokeapi.co/api/v2/pokemon/$numero"

    val pokemonRequest: StringRequest = StringRequest(
        Request.Method.GET,
        url,
        { response ->
```

```
            try {
                val jObj: JSONObject = JSONObject(response)
                val name: String = jObj.getString("name")
                val id: Int = jObj.getInt("id")
                val image: String = jObj.getJSONObject("sprites")
                    .getJSONObject("other")
                    .getJSONObject("official-artwork")
                    .getString("front_default")

                val tipos: ArrayList<String> = ArrayList()
                val types = jObj.getJSONArray("types")
                for (i in 0 until types.length()) {
                    tipos.add(types.getJSONObject(i).getJSONObject("type").getString("name"))
                }

                onResult(Pokemon(name = name, number = id, imagen = image, tipos = tipos))
            } catch (e: JSONException) {
                e.printStackTrace()
                onResult(null)
            }
        },
        { onResult(null) }
    )
    queue.add(pokemonRequest)
}
```

5.3. Buscador.kt

5.3.1. Ubicación

`com.example.pokedexkotlin`

Descripción general:
La clase `Buscador` es la única `Activity` de la app. Dibuja la pantalla con **Jetpack Compose**: la imagen, el nombre, los tipos, el campo del número y el botón de búsqueda.

5.3.2. Funciones

onCreate **Descripción:** Activa el modo de pantalla completa (*edge to edge*) y carga la pantalla `PokedexApp` dentro del tema de la app.

Código comentado:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContent {
        PokedexKotlinTheme {
            Surface(

            ){
                PokedexApp()
            }
        }
    }
}
```

NumberTextFild **Descripción:** Campo de texto de una sola línea que abre directamente el teclado numérico.

Código comentado:

```
@Composable
fun NumberTextFild(label : String, value: String, onValueChange : (String)-> Unit, modifier:
     Modifier = Modifier){
    TextField(
        label = {
            Text(
                text = label
            )
        },
        value = value,
        onValueChange = onValueChange,
        singleLine = true,
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        modifier = modifier
    )

}
```

PokedexApp **Descripción:** Pantalla principal. Guarda el estado con `remember` y `mutableStateOf`, valida el número, llama a la `API` y actualiza la imagen, el nombre y los tipos con la respuesta. Mientras espera, el botón se desactiva y muestra «Buscando...».

Código comentado:

```
@Composable
fun PokedexApp(modifier: Modifier = Modifier) {
    var pokemonName by remember{ mutableStateOf("Nombre del Pokémon")}
    var pokemonTipos by remember{ mutableStateOf("")}
    var pokemonNumber by remember{ mutableStateOf("")}
    var pokemonImage by remember{ mutableStateOf("")}
    var buscando by remember{ mutableStateOf(false)}
    val ct : Context = LocalContext.current

    Column(
        modifier = Modifier
            .statusBarsPadding()
            .padding(horizontal = 40.dp)
            .safeDrawingPadding()
            .fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ){
        AsyncImage(
            model = pokemonImage,
            contentDescription = "Imagen del Pokémon",
            contentScale = ContentScale.Crop,
            error = painterResource(R.drawable.ic_launcher_foreground),
            placeholder = painterResource(R.drawable.ic_launcher_background),
            modifier = Modifier
                .size(220.dp)
                .clip(CircleShape)
        )
        Text(
            text = pokemonName,
            style = MaterialTheme.typography.displaySmall
        )
        Text(
            text = pokemonTipos,
            style = MaterialTheme.typography.titleMedium,
            modifier = Modifier.padding(bottom = 24.dp)
        )
        NumberTextFild(
```

```
            label = "Número de Pokédex (1-1025)",
            value = pokemonNumber,
            onValueChange = {pokemonNumber = it},
            modifier = Modifier.padding(bottom = 32.dp)
                .fillMaxWidth()
        )
        Button(
            enabled = !buscando,
            onClick = {
                val idBuscado : Int? = pokemonNumber.trim().toIntOrNull()
                if (idBuscado == null || idBuscado < 1) {
                    pokemonName = "Escribe un número válido"
                    pokemonTipos = ""
                    pokemonImage = ""
                } else {
                    buscando = true
                    pokemonName = "Buscando..."
                    API().buscarPokemon(ct, idBuscado) { pokemon ->
                        buscando = false
                        if (pokemon != null) {
                            pokemonName = "#${pokemon.number} " + pokemon.name.replaceFirstChar {
 it.uppercase() }
                            pokemonTipos = pokemon.tipos.joinToString(" · ") { it.
replaceFirstChar { c -> c.uppercase() } }
                            pokemonImage = pokemon.imagen
                        } else {
                            pokemonName = "No encontrado"
                            pokemonTipos = ""
                            pokemonImage = ""
                        }
                    }
                }
            }
        ) {
            Text(
                text = if (buscando) "Buscando..." else "Buscar Pokémon"
            )
        }
    }
}
```

GreetingPreview **Descripción:** Vista previa de la pantalla para el editor de Android Studio.

Código comentado:

```
@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    PokedexKotlinTheme {
        PokedexApp()
    }
}
```

6. Mejoras futuras

- Buscar también por **nombre**.
- Cambiar el **color de fondo** según el tipo principal, como en la versión en Java.
- Cambiar la **imagen de reserva** (ahora es el robot de Android) por una Pokéball.

- Pasar la lógica a un `ViewModel` con `StateFlow` y las peticiones a **Retrofit** con corrutinas.
- Añadir una **lista con todos los Pokémon** con `LazyVerticalGrid`.